

Portfólio 5 – Sistema de Consulta de Produtos

Uma pequena loja virtual de eletrônicos precisa de uma página de consulta de produtos para o seu setor de atendimento. O atendente digita o código de um produto e o sistema exibe as informações básicas desse produto.

Desenvolva um módulo simples de consulta usando Spring MVC, NetBeans e MySQL.

Requisitos funcionais:

A aplicação deve possuir uma página inicial com:

- Um campo para informar o ID do produto (por exemplo, um Long ou int).
- Um botão para buscar o produto.

Ao clicar em “Buscar”:

- O sistema deve enviar a requisição para um Controller Spring MVC.
- O Controller deve acessar o banco de dados MySQL, buscar o produto pelo ID informado e:
 - Se encontrar o produto:
 - Redirecionar para uma página de resultado exibindo:
 - ID
 - Nome do produto
 - Descrição
 - Preço
 - Se não encontrar:
 - Exibir uma mensagem de “Produto não encontrado”.

O banco de dados deve conter uma tabela produto com, pelo menos, os campos:

- id (chave primária, numérica)
- nome (texto)
- descricao (texto)
- preco (decimal)

A aplicação deve seguir o padrão Spring MVC, usando:

- Um Controller para tratar as requisições.
- Uma classe de modelo Produto.
- Uma camada de acesso a dados (DAO ou repositório simples).
- Páginas JSP para exibição (View), se estiver usando Spring MVC com JSP.

2. Orientações teóricas para os alunos – O que é Spring MVC?

Spring MVC é um framework do ecossistema Spring voltado para aplicações web, que implementa o padrão Model-View-Controller de forma mais estruturada e produtiva do que o uso direto de Servlets e JSP.

No Spring MVC:

Model (Modelo)

- Representa os dados da aplicação (por exemplo, a classe Produto).
- Pode ser enriquecido com regras de negócio, validações, etc.

View (Visão)

- Normalmente são páginas JSP ou outros mecanismos de template, que recebem dados do Model e exibem ao usuário.

Controller (Controlador)

- Classes Java anotadas com `@Controller` que:
- Recebem as requisições HTTP (GET, POST).
- Processam os parâmetros.
- Chamam a camada de serviço / DAO.
- Retornam o nome da View que será renderizada, junto com os dados.

Principais características do Spring MVC:

- Uso intenso de anotações, que deixam o código mais enxuto (por exemplo, `@Controller`, `@GetMapping`, `@PostMapping`).
- Inversão de Controle (IoC) e Injeção de Dependência – o próprio Spring gerencia objetos, DAOs, serviços, etc.
- Facilita o mapeamento de URLs para métodos Java sem precisar editar `web.xml` manualmente.
- Integração natural com bibliotecas para acesso a banco de dados (JDBC, JPA).

3. Diferenças Spring MVC x MVC tradicional com Servlets

3.1. Estrutura do MVC tradicional (Servlet + JSP)

No MVC tradicional:

Controller = Servlet

- Você cria uma classe que estende `HttpServlet`.
- Implementa `doGet`, `doPost`.
- Usa `request.getParameter()`, `request.setAttribute()` e `RequestDispatcher.forward()` para passar dados para a JSP.
- Configura a URL do servlet em `web.xml` (ou anotação `@WebServlet`).

Model

- Classes Java simples que representam entidades (ex. Produto).
- DAOs criados “na mão” e instanciados dentro do servlet.

View = JSP

- Renderiza as informações vindas de `request.setAttribute()`.

3.2. Estrutura do Spring MVC

No Spring MVC:

Controller

- Não precisa estender `HttpServlet`.
- É uma classe simples com a anotação `@Controller` (ou `@RestController` em APIs).
- Métodos anotados com `@GetMapping`, `@PostMapping`, `@RequestMapping` tratam as URLs.
- Os parâmetros da requisição podem vir como argumentos do método (`@RequestParam`, `@PathVariable`, etc.).
- O retorno do método é, normalmente, uma `String` com o nome da View e um objeto `Model`.

Model

- Classes simples (POJOs), muito parecido com o MVC tradicional.
- DAOs ou Repositories podem ser gerenciados pelo Spring com `@Repository` e injetados com `@Autowired`.

View

- JSP ou outros templates, configurados por um ViewResolver do Spring.
- Em vez de trabalhar com `request.setAttribute()` diretamente, o Controller usa um objeto Model para enviar dados para a View.

Principais diferenças práticas:

Configuração das rotas:

- Servlet MVC: rotas configuradas no `web.xml` ou `@WebServlet`.
- Spring MVC: rotas declaradas com anotações diretamente nos métodos do Controller.

Controle de dependências:

- Servlet MVC: você instancia DAOs e serviços com `new` dentro dos Servlets.
- Spring MVC: o Spring cria e gerencia esses objetos, e você os recebe com `@Autowired`.

Organização do código:

- Servlet MVC: lógica costuma ficar concentrada em Servlets, que tendem a crescer demais.
- Spring MVC: incentiva a separação de camadas (Controller, Service, Repository), deixando a aplicação mais organizada e testável.

Produtividade:

Spring MVC evita muito código repetitivo, reduz configuração manual e oferece diversas funcionalidades prontas (validação, binding de formulários, etc.).

Dicas importantes:

- O Netbeans já possui o framework Spring MVC. Selecione-o ao criar sua aplicação.
- Faça uso de uma ferramenta de IA para dar suporte na construção dessa aplicação. O código deve ser comentado. Entregue toda a aplicação zipada juntamente com o script da tabela.